

Facultad de Ciencias Puras

Carrera de Ingeniería Informática

INF780 – Verificación y Validación de Software

TAREA 4

Pruebas de Rendimiento con Apache JMeter sobre una API REST

Nombre: Lucy Zoraida Beltran Tarqui

1. Introducción

Las pruebas de rendimiento permiten evaluar el comportamiento de un sistema cuando es sometido a diferentes niveles de carga. A diferencia de las pruebas funcionales, cuyo objetivo es verificar que la aplicación responda correctamente, las pruebas de rendimiento buscan medir la capacidad de respuesta, estabilidad y escalabilidad del sistema.

En esta práctica se utilizó Apache JMeter para ejecutar pruebas de carga, estrés y picos sobre una API REST desarrollada con NestJS y PostgreSQL. Durante las pruebas se analizaron métricas como el tiempo de respuesta, throughput, porcentaje de errores y percentiles, con el propósito de identificar el comportamiento de la API bajo diferentes niveles de concurrencia.

Objetivos

Objetivo general

Aplicar pruebas de rendimiento utilizando Apache JMeter sobre una API REST para evaluar su comportamiento bajo diferentes niveles de carga.

Objetivos específicos

- Instalar y configurar Apache JMeter.
- Diseñar planes de prueba para distintos escenarios.
- Ejecutar pruebas Smoke, Carga, Estrés y Picos.
- Analizar métricas de rendimiento.
- Identificar el punto de saturación de la API.
- Documentar los resultados obtenidos.

2. Entorno de pruebas

Las pruebas de rendimiento fueron ejecutadas en un entorno local utilizando la API REST desarrollada en NestJS con base de datos PostgreSQL. El objetivo de documentar el entorno es permitir la reproducción de los resultados obtenidos.

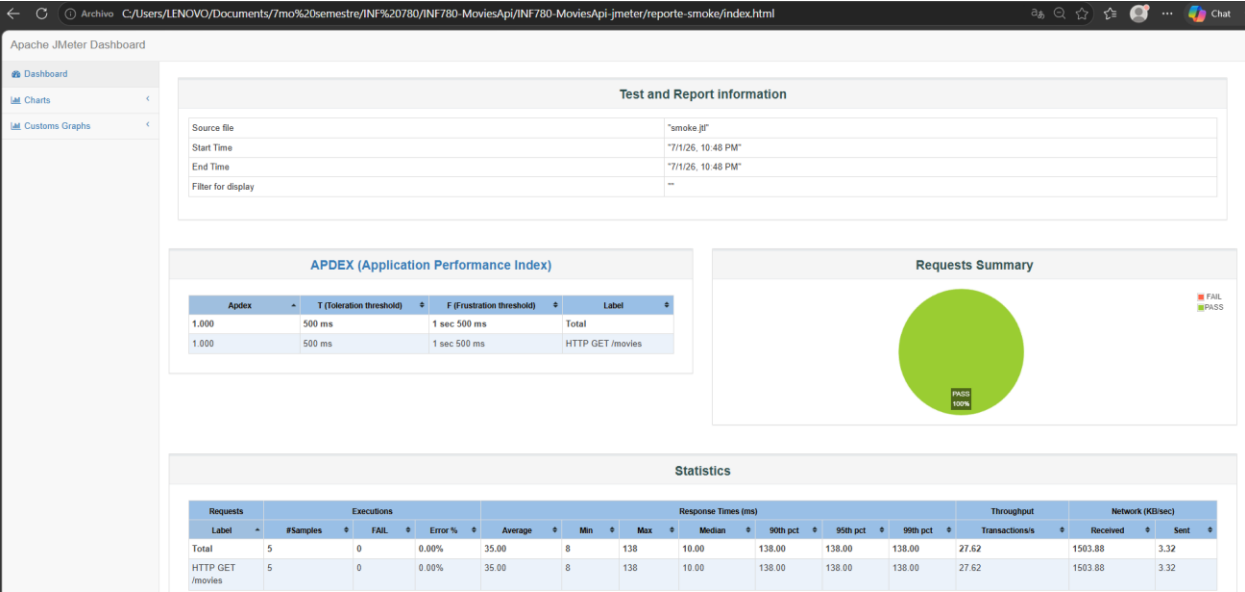
Componente	Especificación
------------	----------------

Sistema Operativo	Microsoft Windows 11 Home (Versión 10.0.26200)
Equipo	Lenovo LOQ 15ARP9
Procesador	AMD Ryzen 5 7235HS @ 3.20 GHz (4 núcleos, 8 hilos)
Memoria RAM	12 GB
Arquitectura	x64
Java	OpenJDK 17.0.19 (Temurin)
Apache JMeter	5.6.3
Framework Backend	NestJS
Base de Datos	PostgreSQL
API evaluada	Movies API (localhost:3000)

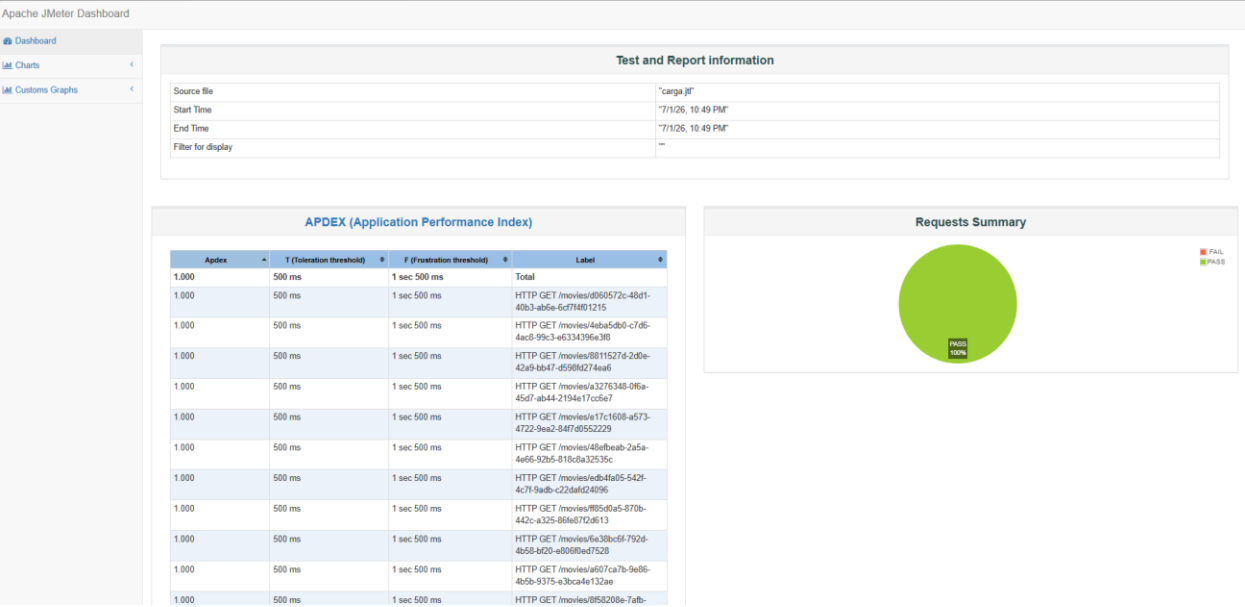
3. Diseño de las pruebas

Escenario	Usuarios	Ramp-Up	Loops	Endpoints	Aserciones
Smoke	1	1 s	5	GET /movies	Ninguna
Carga	50	30 s	10	GET /movies, GET /movies/{id}, POST /movies	Ninguna
Estrés	100, 200 y 400	30 s	10	GET /movies, GET /movies/{id}, POST /movies	Ninguna
Picos	200	5 s	10	GET /movies, GET /movies/{id}, POST /movies	Response Assertion y Duration Assertion (800 ms)

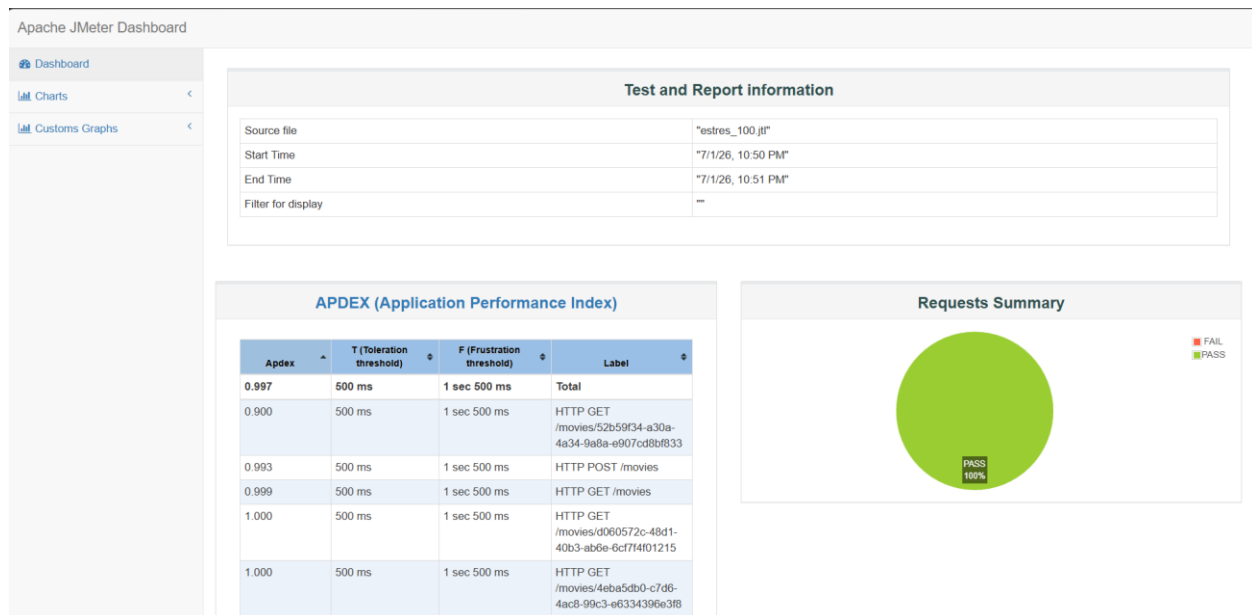
4.1 Prueba Smoke



Reporte carga



Reporte estrés 100



5. Análisis comparativo

5.1 Tabla Comparativa de Resultados

Escenario	Usuarios Concurrentes	Throughput (TPS)	Tiempo Medio (ms)	Porcentaje de Error
Smoke	1	27.62	35 ms	0.00%
Carga	50	38.10	19 ms	0.00%
Estrés 100	100	70.10	111 ms	0.00%
Estrés 200	200	62.10	1,773 ms	0.00%
Estrés 400	400	33.00	10,737 ms	0.00%
Picos (Spike)	200 (en 5s)	21.60	8,803 ms	99.17% (SLA duration fail)

5.2 Punto de Saturación

Hasta 100 usuarios concurrentes: El sistema escala eficientemente, alcanzando su rendimiento óptimo con un throughput máximo de 70.1 TPS y tiempos de respuesta muy rápidos (media de 111 ms).

De 100 a 200 usuarios concurrentes (Punto de Saturación): El throughput empieza a decaer levemente a 62.1 TPS y el tiempo medio de respuesta sube exponencialmente de 111 ms a 1,773 ms, marcando la saturación de procesamiento.

A partir de 400 usuarios concurrentes: El throughput colapsa a 33.0 TPS y la latencia sube a 10.7 segundos, entrando el sistema en congestión por intercambio de contexto.

5.3 Cuellos de Botella

Serialización de Payloads Voluminosos: El endpoint GET /movies descarga toda la colección de películas sin paginar, bloqueando el event loop de Node.js al serializar JSON y agotando la memoria.

Cola en Conexiones a Base de Datos: A partir de los 200 usuarios concurrentes, el pool de conexiones de TypeORM/PostgreSQL satura la base de datos, encolando las peticiones pendientes.

6. Conclusiones y recomendaciones

6.1 Conclusiones

La API demuestra excelente estabilidad bajo cargas moderadas (hasta 100 usuarios concurrentes).

El servidor demostró resiliencia al no arrojar errores 5xx de caída de sistema durante las pruebas de estrés.

Las ráfagas repentinas (Prueba de Picos) degradan la latencia debido al encolamiento rápido de peticiones, superando el SLA de 800 ms.

6.2 Recomendaciones de Mejora

Paginación: Implementar paginación obligatoria (limit y offset) en GET /movies para reducir el tamaño del payload de respuesta.

Caché (Redis): Almacenar las respuestas de las consultas en memoria caché para evitar llamadas repetidas a la base de datos.

Índices en BD: Configurar índices en PostgreSQL sobre las columnas de búsqueda más utilizadas.

Optimización del Pool: Incrementar el tamaño máximo del pool de conexiones en TypeORM para agilizar las peticiones concurrentes a la base de datos.

Configuración JMeter: En escenarios de alto volumen de datos, usar postprocesadores de JMeter para descartar la respuesta en RAM (prev.setResponseData(new byte[0])) y evitar errores de memoria.

7. Anexos

7.1 Listado de Archivos de Prueba (.jmx)

smoke.jmx: Comprobación inicial (1 usuario, 5 loops).

carga.jmx: Carga típica (50 usuarios, ramp-up 30s, 10 loops).

estres.jmx (y estres_100.jmx): Estrés bajo (100 usuarios, ramp-up 30s, 10 loops).

estres_200.jmx: Estrés medio (200 usuarios, ramp-up 30s, 10 loops).

estres_400.jmx: Estrés crítico (400 usuarios, ramp-up 30s, 10 loops).

picos.jmx: Ráfaga (200 usuarios, ramp-up 5s, 10 loops) con aserciones de duración (800 ms).

7.2 Ubicación de Resultados (.jtl) y Reportes HTML

Todos los archivos se encuentran en la subcarpeta del entregable INF780-MoviesApi-jmeter/:

Archivo de Datos: ids.csv

Resultados JTL: smoke.jtl, carga.jtl, estres_100.jtl, estres_200.jtl, estres_400.jtl, picos.jtl.

Reportes HTML (abrir index.html dentro de cada carpeta):

reporte-smoke/

reporte-carga/

reporte-estres-100/

reporte-estres-200/

reporte-estres-400/

reporte-picos/